

Федеральное государственное автономное образовательное учреждение высшего
образования



**«Московский государственный технический университет
имени Н.Э. Баумана»**

(национальный исследовательский университет)

(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

О т ч е т

по лабораторной работе № 3

Название лабораторной работы: Создание консольного приложения

Дисциплина: Алгоритмизация и программирование

Студент гр. ИУ6-23Б _____ **С.М. Соболев**
(Подпись, дата) (И.О. Фамилия)

Преподаватель _____ **О.А. Веселовская**
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

Часть 1. Изучить среду разработки консольных приложений на языке программирования C#.

Цель работы: изучение среды разработки для языка программирования C#.

Выполнение: диаграмма структуры программы изображена на рисунке 1.

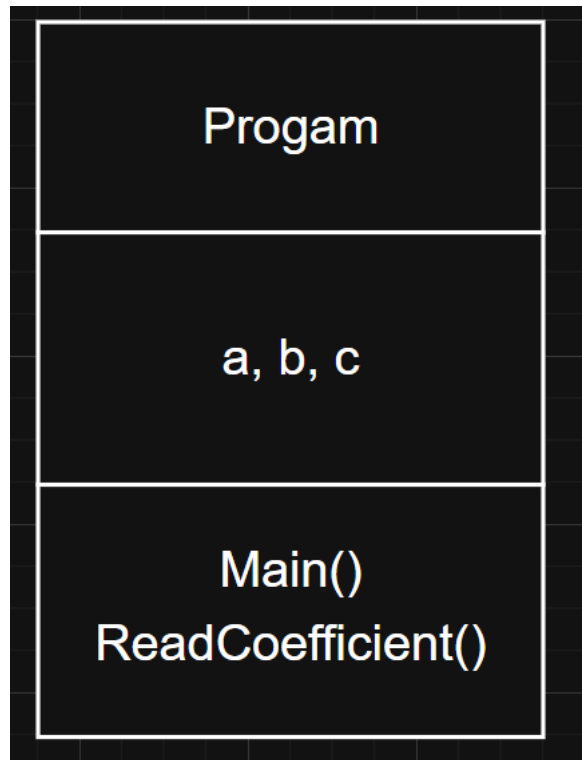


Рисунок 1 – Диаграмма классов

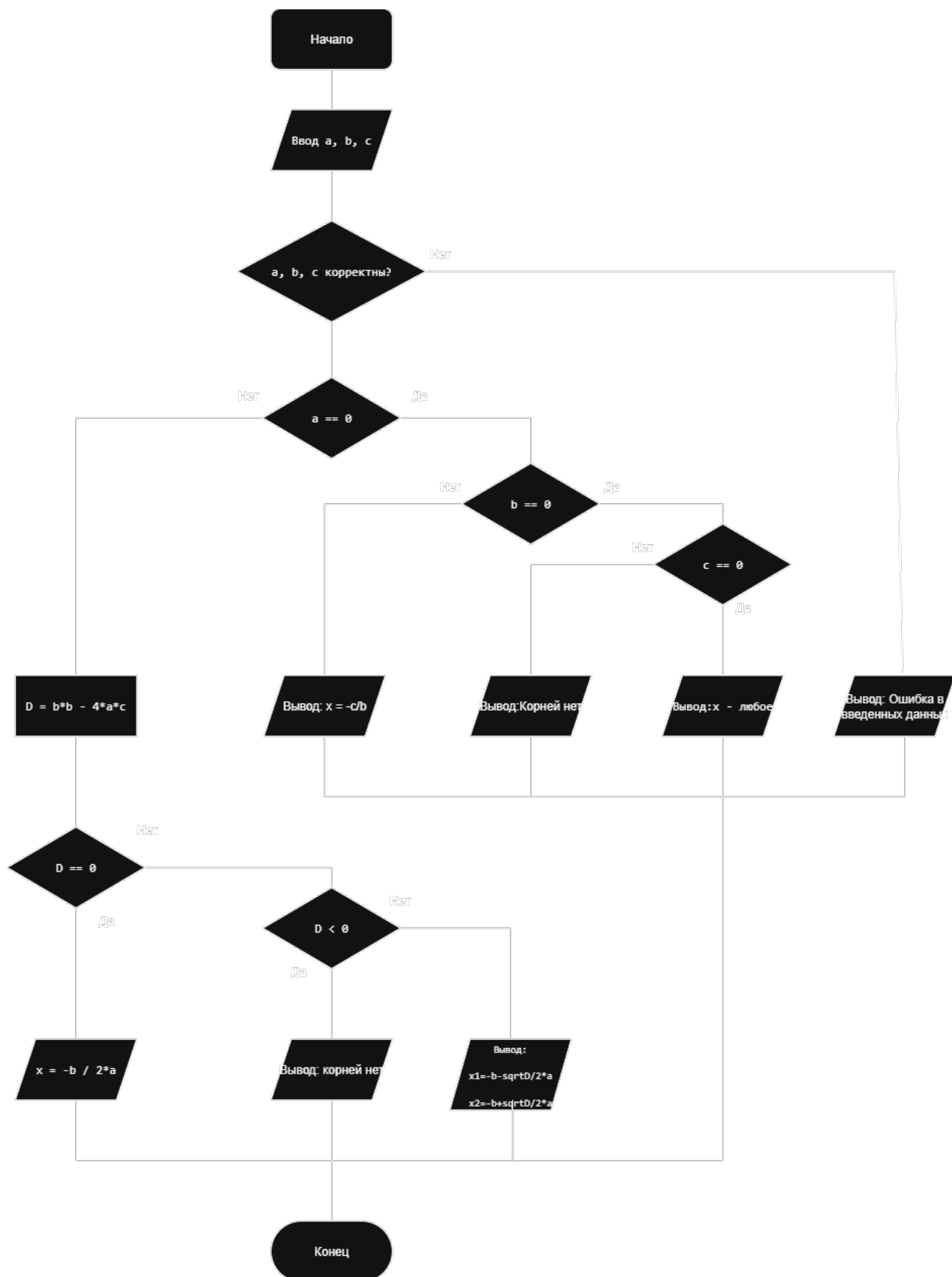


Рисунок 2 – Схема алгоритма

```

using System;
namespace Lab3Project
{
    Ссылка: 0
    class Program
    {
        Ссылка: 0
        static void Main(string[] args)
        {
            try
            {
                double a = ReadCoefficient("Введите коэффициент a: ");
                double b = ReadCoefficient("Введите коэффициент b: ");
                double c = ReadCoefficient("Введите коэффициент c: ");
                if (a == 0)
                {
                    Console.WriteLine("\nКоэффициент 'a' равен 0. Уравнение является линейным (bx + c = 0).");
                    if (b != 0)
                    {
                        double linearX = -c / b;
                        Console.WriteLine($"Корень уравнения: x = {linearX}");
                    }
                    else
                    {
                        if (c == 0)
                            Console.WriteLine("Уравнение имеет бесконечное множество решений (0 = 0).");
                        else
                            Console.WriteLine("Уравнение не имеет решений.");
                    }
                }
                return;
            }
            double discriminant = Math.Pow(b, 2) - 4 * a * c;
            Console.WriteLine($"Дискриминант D = {discriminant}");
            if (discriminant > 0)
            {
                double x1 = (-b + Math.Sqrt(discriminant)) / (2 * a);
                double x2 = (-b - Math.Sqrt(discriminant)) / (2 * a);
                Console.WriteLine($"Уравнение имеет два различных корня: \nx1 = {x1} \nx2 = {x2}");
            }
            else if (discriminant == 0)
            {
                double x = -b / (2 * a);
                Console.WriteLine($"Уравнение имеет один корень (два совпадающих): \nx = {x}");
            }
            else
            {
                Console.WriteLine("Действительных корней нет, так как дискриминант меньше нуля.");
            }
        }
        catch (FormatException)
        {
            Console.WriteLine("Ошибка: Введены некорректные данные.");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Ошибка: {ex.Message}");
        }
    }
}

```

Рисунок 3 – Код основной программы

Тестовые данные и результаты тестирования изображены на Таблице 1.

a	b	c	Вывод
0	0	0	Уравнение имеет бесконечное множество решений
0	0	1	Уравнение не имеет решений
0	5	2.5	Уравнение имеет один корень $x = -0.5$
1	5	-6	Уравнение имеет два различных корня: $x_1 = -6$ $x_2 = 1$
Jjj			Ошибка: Введены некорректные данные.

```

Введите коэффициент a: 2
Введите коэффициент b: 3
Введите коэффициент c: 1

Дискриминант D = 1
Уравнение имеет два различных корня:
x1 = -0,5
x2 = -1

```

Рисунок 4 – Результат работы программы

Вывод: в ходе лабораторной работы было изучено создание консольных приложений на языке C++, разработана программа для нахождения корней квадратного уравнения с обработкой ошибок при вводе нечисловых данных и учтены случаи отсутствия корней. Программа была отлажена и протестирована на разных наборах данных.

Часть 2. Диагностические сообщения компилятора и средства отладки

Цель работы: изучить диагностические сообщения компилятора и средства отладки.

Выполнение: точка остановки и процесс отладки изображены на рисунке 6.

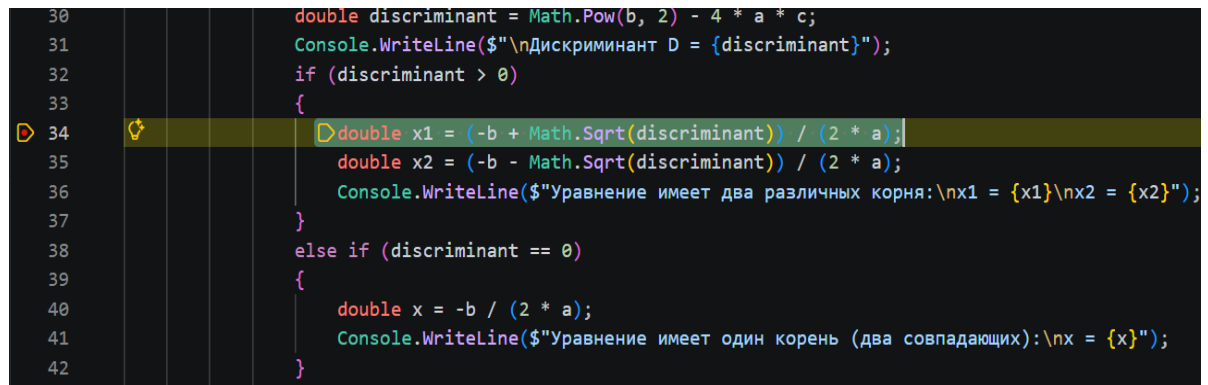


Рисунок 5 – Точка остановки и процесс отладки

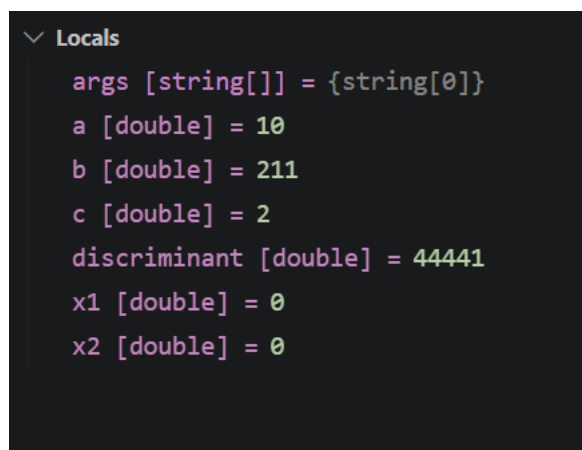


Рисунок 6 – Просмотр промежуточных значений

Вывод: в ходе лабораторной работы было изучено использование диагностических сообщений компилятора и средств отладки в IDE. На примере программы для решения квадратного уравнения были применены средства отладки для установки контрольных точек и анализа промежуточных результатов выполнения программы.